mb-free___faults-raise

An AgentMPI run analysis

Abstract

This is the `RAISE` point of the four-policy barrier-failure sweep in `bench_faults` (`experiments/microbench.py`): eight ranks, rank 3 returns immediately to simulate a crashed agent session, and the seven survivors enter a barrier with a 15 s deadline. `RAISE` is the default `BarrierPolicy` and is supposed to raise `AmpiTimeout` at the deadline. What the trace shows is that the policy was never consulted. `algorithms.barrier` switches to the arrival-counting `central` algorithm only for `PROCEED`, `SHRINK` and `REVOKE`; `RAISE` keeps the `dissemination` default, and the dissemination barrier has no deadline branch of its own — it simply threads the caller's timeout into every send and receive. So no `barrier.timeout` was emitted, no `ft.declare_failed` was recorded, no `coll.barrier` was logged, and the run's fabric ends with an empty `failures` table and all eight members `active`. What did happen is a three-round collapse: of the 24 messages a dissemination barrier needs at $p = 8$, only 17 were sent and only 10 were received, and the seven survivors blocked at staggered depths in the recursion — rank 4 after one send, ranks 5 and 6 after two, ranks 0, 1, 2 and 7 after three. Every one of them then raised the identical error, `ERR_TIMEOUT: receive timed out [...source=4 ...]`, naming a rank that was alive. The single most important thing to take away is the gap between the run and its own instrumentation: the benchmark scores this row `detected_correctly: false`, and yet `metrics.json` reports `degraded: false`, `failed_ranks: []` and `failures: 0`, the generated findings list reads "nothing anomalous was detected mechanically", and the viewer's FAILURES tile reads **0**. Of the four policies this is the one that failed hardest, and it is the one whose dashboard looks healthiest.

1 What this run is

property	value
run	`mb-free__faults-raise`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	45
wall time	15.27 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	17	17 eager, 0 rendezvous
tokens sent	17	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

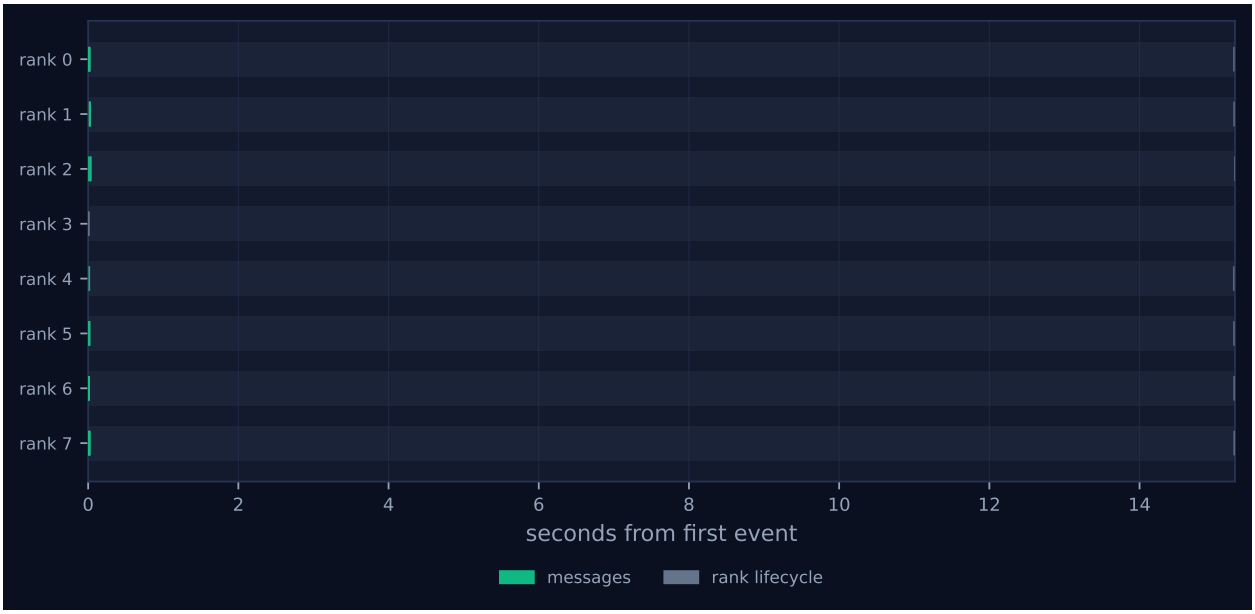

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

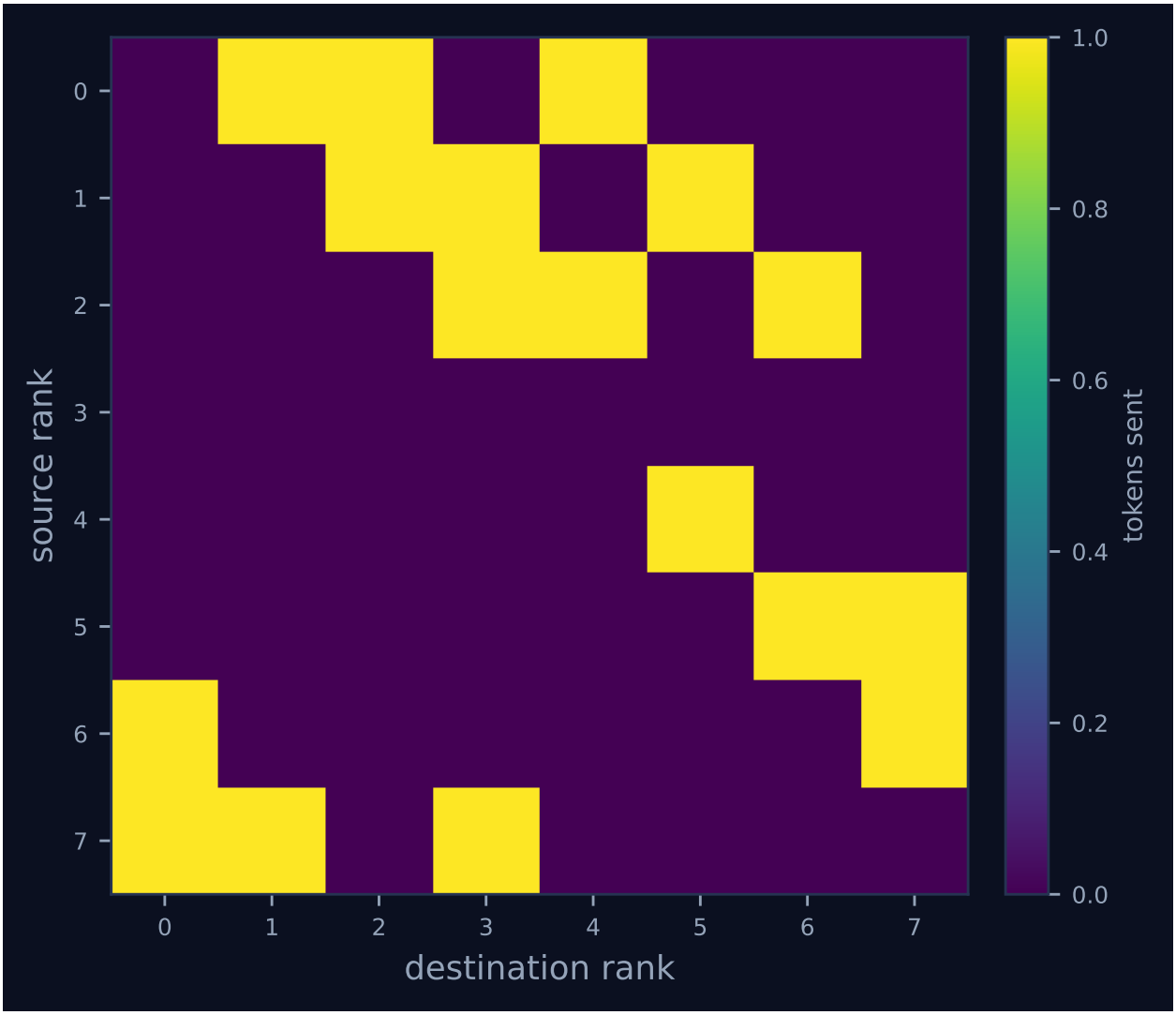

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	3	2	3	0.0	finalized
1	0.00	0.0	0	0	3	2	3	0.0	finalized
2	0.00	0.0	0	0	3	2	3	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	1	0	1	0.0	finalized
5	0.00	0.0	0	0	2	1	2	0.0	finalized
6	0.00	0.0	0	0	2	1	2	0.0	finalized
7	0.00	0.0	0	0	3	2	3	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 has two vertical columns and nothing between them: a green cluster hard against $t = 0$ and a grey column at $t \approx 15.25\,\mathrm{s}$. The green marks are the barrier's messages, all 17 of which were sent inside the first 32 ms; the grey marks are `rank.finalize`. There are no bars, because there is no work. `metrics.json` records `executors: {none: 8}`, zero agent calls, zero tokens and $0.0000, so the headline table's *achieved parallelism* 0.00× and *idle fraction 100.0%* describe the benchmark's design and are not findings about the protocol. `bench_faults`'s `rank_main` calls `comm.barrier` and nothing else; a rank with no task cannot have a busy fraction. Read both rows as "not applicable". The 15.271 s wall clock is likewise a constant: it is `-detect-timeout` (15.0 s, from `results/microbench/free-faults.json`) plus 271 ms of process start-up and SQLite writes.

Rank 3's lane holds a single tick at the far left — `rank.init` at $t = 0.007\,\mathrm{s}$ and `rank.finalize` at $t = 0.010\,\mathrm{s}$, three milliseconds apart — and then nothing. That is the injected fault in its entirety.

The interesting structure is compressed into the first 32 ms and is invisible at the figure's scale, so the per-rank table in `generated.tex` is the better place to read it. Its *sent* and *recv* columns are not uniform, and the pattern is exact:

rank	sent	recv	where it stopped
3	0	0	the victim; never entered the barrier
4	1	0	blocked in round 0, waiting on rank 3
5	2	1	blocked in round 1, waiting on rank 3
6	2	1	blocked in round 1, waiting on rank 4
0, 1, 2, 7	3	2	blocked in round 2, waiting on ranks 4, 5, 6 and 3

A dissemination barrier at $p = 8$ runs $\lceil \log_2 8 \rceil = 3$ rounds; in round k rank r sends to $(r + 2^k) \bmod 8$ and receives from $(r - 2^k) \bmod 8$. Rank 4's round-0 source is rank 3, so rank 4 sends once and then

blocks immediately. Rank 5's round-1 source is rank 3 and rank 6's is rank 4 — already stalled — so both get two sends in. The remaining four complete two full rounds and block in round 2 on sources 4, 5, 6 and 3 respectively. The death of one rank propagates to the entire population in exactly the three rounds the algorithm takes, which is the same logarithmic structure that makes the barrier cheap. Summing the column gives $7 + 6 + 4 = 17$ sends against the 24 a complete barrier would need, and 10 receives.

Figure 2 shows the same thing spatially and is the clearest single artefact in the run. Row 3 is empty across its whole width: the dead rank sent nothing. Rows 0, 1 and 2 each carry three cells, rows 5 and 6 two, row 4 one — the staggered blocking depth, read off as row length. Column 3 carries three cells, from ranks 2, 1 and 7: three messages delivered into a dead rank's mailbox, one per round. Seven of the seventeen messages were never received. Three went to rank 3 and four went to ranks 4, 5 and 6, which were alive but had already stopped reading. Nothing in the run notices either category.

The viewer confirms the reading and adds the punchline. Its stats row shows MESSAGES 17 and FAILURES 0. There is no COLLECTIVES panel at all, because no `coll.*` event was ever emitted. Its rank table lists all eight ranks as `finalized`, `alive: no`, `suspected: -`, with rank 3 indistinguishable from the seven that outlived it by fifteen seconds. The legend counts *messages 27, rank lifecycle 16* and offers no failure or recovery colour, so the dashboard for this run renders in green and grey only.

7 Interpretation

7.1 The policy was never applied

The central fact is a control-flow one and it is worth tracing precisely, because the run name promises something the run does not contain.

`algorithms.barrier` begins:

```
if policy in (BarrierPolicy.PROCEED, BarrierPolicy.SHRINK, BarrierPolicy.REVOKE):
                                algorithm = "central"
```

`RAISE` is not in that tuple, so `algorithm` keeps the signature's `"dissemination"` default that `bench_faults` did not override. The dissemination branch is a plain loop of `comm._csend` and `comm._crecv` calls with `timeout=timeout` threaded through, and it contains no reference to `policy` whatsoever. The only code that reads the policy is `_central_barrier`, which this run never enters. So the `AmpiTimeout` that every survivor raised came from `Communicator.recv`'s own deadline, not from the barrier's policy branch, and the run would have been byte-for-byte the same had the policy argument been omitted.

The trace proves this rather than merely being consistent with it. If `_central_barrier` had run, it would have emitted `barrier.timeout` before raising — the emit precedes the `raise` in the source — and it would have called `_record_collective`, leaving a row in the fabric's `collectives` table. Querying `runs/mb-free/faults-raise/fabric.sqlite` shows that table *empty*, alongside an empty `failures` table, `comms.generation = 0`, `comms.revoked = 0`, and all eight `comm_members` rows `active`. The communicator ended the run in exactly the state it started in.

There is a further consequence that makes `RAISE` and `WAIT` synonyms. Even on the path where the policy *is* read, the two share a branch: `if policy is BarrierPolicy.RAISE or policy is`

`BarrierPolicy.WAIT: raise AmpiTimeout(...)`. Grepping `src/agentmpi` finds `BarrierPolicy.WAIT` exactly once, in that line. So there is no code path anywhere in AgentMPI on which `RAISE` and `WAIT` behave differently, and the two runs in this sweep bear it out: `mb-free__faults-wait` has the same 45 events, the same 17-send and 10-receive counts, the same per-rank profile and an identical 17-edge communication matrix, differing only in the millisecond ordering and 11 ms of wall clock. The sweep's four policies produce three distinct behaviours.

7.2 What the run does establish

Read as a measurement of MPI-like semantics rather than of a policy, this run is informative, and I would argue it is the sweep's most important negative result.

One dead rank at $p = 8$ costs all seven survivors. The blocking is total, not partial, and the trace shows the propagation path explicitly. This is the pathology `algorithms.barrier`'s docstring names — "the probability that all p members arrive within any fixed window falls off with p" — observed at the smallest population where it has room to develop. It also generalises badly: at $p = 8$ the failure reaches everyone in 3 rounds, and the round count is $\lceil \log_2 p \rceil$, so a larger population does not dilute the damage, it merely delays it by a round or two.

The timeout is bounded and honoured, and that is genuinely worth something. Every survivor finalised between 15.254 s and 15.270 s. Nothing hung. Every wait in this trace is the value of a named constant, which is the property `real-tr-p16-full`'s analysis credits the protocol with and which is not free — an MPI barrier with a dead peer hangs forever.

But the diagnosis is wrong, and wrong in a specific and damaging way. The recorded error is `ERR_TIMEOUT: receive timed out [ctx=0 rank=0 source=4 tag='_ampi:barrier:0:1:2' waited_s=15.0]`. Rank 0's round-2 source is $(0 - 4) \bmod 8 = 4$, and rank 4 was alive throughout — it finalised normally at $t = 15.254$ s. The error names an innocent rank, and it must, because a dissemination barrier gives each rank visibility of exactly three peers and rank 3 is not one of rank 0's. The benchmark's `detected_correctly` field encodes this directly: it compares each survivor's `detected` list against the victim list `[3]`, and because the fallback `ampi.get_failed(comm)` returns an empty tuple against an empty `failures` table, every survivor reports an empty list. The published row reads `"detected_correctly": false`.

The practical weight of this is that the failure is not merely undiagnosed but *unrecoverable-from*. `ft.shrink` begins with `_agree_survivors`, which computes the survivor set from `get_failed` — empty here — unioned with whatever `ft.health`'s lease detector suspects; `ft.agree` computes `expected` the same way. With an empty `failures` table, both would reconstruct a group that still contains rank 3 and block on it again. A harness that catches the `AmpiTimeout` from a `RAISE` barrier and tries to recover has nothing to recover with.

7.3 The flagged findings, and the one that is missing

The generated list says: *"Nothing anomalous was detected mechanically."* That is the finding worth discussing, because it is true as defined and badly wrong as read.

The mechanical checks fire on incomplete collectives, on cost-model disagreements, on failed ranks reported by `job.finish`, and on `ok: false`. This run trips none of them, and each miss has the same root cause: *nothing was recorded.* `Analysis.degraded` is `bool(failed_ranks) or ok is False or bool(incomplete_collectives)`. There are no incomplete collectives because there are

no collectives — `n_collectives: 0`, and the collective section of `generated.tex` reads "This run invoked no collectives" for a run that invoked exactly one. `failed_ranks` is copied from `job.finish`, which lists ranks whose `rank_main` raised, and `bench_faults` catches `ampi.AmpiError` inside `rank_main`, so the job reported `ok: true` with an empty list. `coordination_is_underreported` is `false` here, which is technically correct — there is no completed collective whose time is being undercounted — and misleading in effect, since $7 \times 15.2 = 106$ rank-seconds of blocking inside a barrier are simply absent from every coordination figure, and the flag that would have warned a reader stays down. The `PROCEED` and `SHRINK` siblings, which *did* record their barrier, both raise the flag and both report `degraded: true`.

So the comparison that this run contributes to the sweep is uncomfortable: the two policies that handled the failure well are flagged as degraded, and the two that handled it worst pass every mechanical check. The analysis tooling is not at fault — it can only score what the log contains — but the shape of the miss is worth naming, because a reader scanning 500 runs for trouble would skip this one. I would call it a real observability gap rather than a defect in `analysis.py`: the fix belongs upstream, in having the dissemination barrier record its invocation and its failure the way the central one does.

7.4 Against its siblings, and what a harness author should do

The four runs are the same program with one enum changed, so every difference between them is attributable to the policy:

policy	algorithm	events	wall (s)	msgs	`ft.declare_failed`	absentee named
`wait`	dissemination	45	15.26	17	0	no
`raise`	dissemination	45	15.27	17	0	no
`proceed`	central	39	15.22	0	7	yes
`shrink`	central	60	75.43	0	14	yes, then seven false ones

`PROCEED` detects correctly in 15.195 s, writes one durable failure row, leaves the communicator intact, and hands the harness the absentee list through a normal return. `SHRINK` detects identically and then attempts a repair that breaks: seven concurrent `UPDATE comms SET generation = generation + 1` statements in `ft.shrink_in_place` drive the generation from 0 to 7, the survivors cache six different values, and the `ft.agree` that follows splits across six separate collectives, blocks for its full 60 s, and ends with seven false failure declarations and five ranks returning `true` over `voters: []`.

The recommendation is `PROCEED` wherever a phase's input can legitimately be partial and `REVOKE` plus an explicit out-of-place `ft.shrink` wherever it cannot; and `RAISE` — which is the default — should be treated as unsuitable for any barrier a harness intends to recover from. The evidence from this run is direct. `RAISE` costs the same fifteen seconds as `PROCEED` and buys strictly less: no absentee list, no failure record, an error message naming a live rank, and 17 messages of traffic that `PROCEED` does not send at all. The only thing it provides that `PROCEED` does not is an exception, and an exception carrying the wrong rank number is worse than a return value carrying the right one.

The archive supplies the case that makes this concrete. `real-tr-p16-full` is the only genuine failure among the 500 runs: sixteen ranks, all sixteen in `failed_ranks`, `ok: false`, 7,200 s of wall clock, $0.0726 spent and no output. Six rank roles were never bound to a worker process, and the ten that worked correctly died waiting inside a glossary `allreduce` that could never fold. Its `failures`

table has zero rows and its log has zero `ft.declare_failed` events — the same empty-record state this run ends in, reached the same way, by a collective that blocks on a timeout and records nothing about who failed to arrive. Its own analysis concludes that a shrink over the ten ranks that actually had executors would have finished the job in minutes. (Ten, not twelve: twelve is the count of ranks that emitted a `coll.reduce` record, which is a different set from the set that could do work.)

The caveat this sweep is well placed to add is that *no* choice among these four names would have saved that run. Reading the signatures in `src/agentmpi/algorithms.py`, `policy` is a parameter of `barrier` and of nothing else — `bcast`, `scatter`, `gather`, `allgather`, `reduce`, `allreduce` and `reduce_scatter` take a `timeout` and no policy at all. The collective that killed the p=16 run was an `allreduce`, and an `allreduce` behaves the way this run behaves: it blocks, it times out, and it records nothing about who did not arrive. That is the deeper reading this run supports. The problem `RAISE` exhibits is not really a bad policy choice; it is the default behaviour of every collective in AgentMPI except the central barrier, and it is what a harness gets unless it goes out of its way.

8 Threats to this reading

The round-by-round reconstruction is derived, not logged. No event says which round a message belonged to; I recovered the structure from the internal tags, which encode it as `_ampi:barrier:{generation}:{epoch}:{k}`, and checked the resulting send and receive counts against the per-rank table in `generated.tex`, which matches exactly ($7 + 6 + 4$ sends; $6 + 4$ receives). The mapping from those counts to "rank 5 blocked waiting on rank 3" additionally assumes the dissemination schedule $(r \pm 2^k) \bmod p$ as implemented, which I read from the source. It is a short inference and the arithmetic is over-determined, but it is an inference.

The claim that the failure is unrecoverable-from is a source reading, not an observation. `rank_main` catches the `AmpiTimeout` and returns; it never attempts `ft.shrink` or `ft.agree`, so this trace cannot show what would happen if it did. I am relying on `_agree_survivors` computing its survivor set from `get_failed` plus `ft.health`, and on the `failures` table being empty. The one hole in that reading is `ft.health`: its lease-based detector could in principle have suspected rank 3 independently, since rank 3's lease would eventually expire. Nothing in this run exercises it, and whether it would fire inside a 15 s window depends on the lease length, which this trace does not record.

There are no agents here and nothing in this run bears on agent behaviour. `executors: {none: 8}`, zero agent calls, zero tokens, $0. The 17 messages carry one token each and their digests are all `6b86b273ff34` — the same one-byte payload, seventeen times. The failure modelled is a rank that returns instantly, which is `FAIL_STOP` and only `FAIL_STOP`; `src/agentmpi/ft.py`'s own taxonomy is explicit that the class that matters for agent systems is `FAIL_PLAUSIBLE`, a rank that returns a confident wrong answer, and that no deadline detects it. These four runs exercise one row of a six-row table.

Two counters named `tokens_deferred` disagree across artefacts, and this run is a clean instance of it. `job.finish` reports 10; `metrics.json` reports 0 at the top level and surfaces the 10 as `summary.tokens_unadmitted`. Both are correct under their own definitions — `analysis.py` counts tokens on rendezvous *sends*, of which there were none, while the runtime counts received tokens not admitted to a context, which is all ten receives at one token each — but a reader putting the two files side by side sees the same name with different values. The quantity is trivially

small here and nothing in this reading depends on it; the same collision at a scale that matters is documented in `real-tr-p16-full`, where the two figures are 50,544 and 16,082.

$n = 1$**, but almost nothing here is statistical.** A rerun would shuffle the millisecond ordering of the 17 sends and move the wall clock by tens of milliseconds. It would not change the send and receive counts, which are determined by the dissemination schedule and the victim's identity; it would not change the empty `failures` table or the empty `collectives` table, which are determined by which branch of `algorithms.barrier` runs; and it would not change the identity of the rank named in the error, which is fixed by $(0 - 4) \bmod 8$. The one genuinely schedule-dependent detail is which survivor's error the benchmark happens to report as `example_error`. Where this analysis rests on a number I have tried to say what determines it, and in every case the answer is the algorithm, the configuration, or the fabric — not the run.